sample.ass: Source and Symbol Table

This file shows the source of `tests/src/sample.ass` next to the symbol table the lexer writes for it (`sample.ass.sym`), split into sections. Each section has the source lines first, then the tokens those lines produce.

Generated with `./build/ass run tests/src/sample.ass`.

Summary

Category	Tokens
IDENTIFIER	56
KEYWORD	23
RESERVED_WORD	4
CONSTANT	34
ARITH_OP	7
ASSIGN_OP	25
RELATIONAL_OP	9
LOGICAL_OP	3
RESERVED_OP	5
DELIMITER	38
BRACKET	44
COMMENT	4
INVALID	14
Total	**266**

1. Comments

A single-line `$$` comment and a multi-line `@$$ … @$$` block. Both appear in the table under COMMENT; the block's line breaks are shown as `\n`.

Source (lines 1–5)

```
  1 | $$ Sample ASS-Lang program covering every token in the proposal
  2 | @$$
  3 | Applied Student-Oriented Scripting Language
  4 | Group 6 sample program
  5 | @$$
```

Symbol table

```
ROW:COL    CATEGORY       KIND             LEXEME
1:1        COMMENT        LINE_COMMENT     $$ Sample ASS-Lang program covering every token in the propo
sal
```

```
2:1         COMMENT         BLOCK_COMMENT     @$$\nApplied Student-Oriented Scripting Language\nGroup 6 sa
mple program\n@$$
```

2. Input and Output

Declaration with the `wnum` keyword, then the `read` and `write` statements.

Source (lines 7–9)

```
  7 | wnum age;
  8 | read(age);
  9 | write(age);
```

Symbol table

```
ROW:COL     CATEGORY        KIND              LEXEME
7:1         KEYWORD         KW_WNUM           wnum
7:6         IDENTIFIER      IDENT             age
7:9         DELIMITER       SEMI_COLON        ;
8:1         KEYWORD         KW_READ           read
8:5         BRACKET         LPAREN            (
8:6         IDENTIFIER      IDENT             age
8:9         BRACKET         RPAREN            )
8:10        DELIMITER       SEMI_COLON        ;
9:1         KEYWORD         KW_WRITE          write
9:6         BRACKET         LPAREN            (
9:7         IDENTIFIER      IDENT             age
9:10        BRACKET         RPAREN            )
9:11        DELIMITER       SEMI_COLON        ;
```

3. Declarations, Assignment and Constants

All four data-type keywords and every constant type: INT, FLOAT, CHAR (including an escape), and the boolean constants `True` / `False`.

Source (lines 11–17)

```
 11 | wnum score = 94;
 12 | score = 95;
 13 | dnum ratio = 15.05;
 14 | char grade = 'A';
 15 | char newline = '\n';
 16 | bool passed = True;
 17 | bool failed = False;
```

Symbol table

```
ROW:COL     CATEGORY        KIND              LEXEME
11:1        KEYWORD         KW_WNUM           wnum
11:6        IDENTIFIER      IDENT             score
11:12       ASSIGN_OP       EQ                =
11:14       CONSTANT        INT               94
11:16       DELIMITER       SEMI_COLON        ;
12:1        IDENTIFIER      IDENT             score
12:7        ASSIGN_OP       EQ                =
12:9        CONSTANT        INT               95
12:11       DELIMITER       SEMI_COLON        ;
```

```
13:1        KEYWORD         KW_DNUM          dnum
13:6        IDENTIFIER      IDENT            ratio
13:12       ASSIGN_OP       EQ               =
13:14       CONSTANT        FLOAT            15.05
13:19       DELIMITER       SEMI_COLON       ;
14:1        KEYWORD         KW_CHAR          char
14:6        IDENTIFIER      IDENT            grade
14:12       ASSIGN_OP       EQ               =
14:14       CONSTANT        CHAR             'A'
14:17       DELIMITER       SEMI_COLON       ;
15:1        KEYWORD         KW_CHAR          char
15:6        IDENTIFIER      IDENT            newline
15:14       ASSIGN_OP       EQ               =
15:16       CONSTANT        CHAR             '\n'
15:20       DELIMITER       SEMI_COLON       ;
16:1        KEYWORD         KW_BOOL          bool
16:6        IDENTIFIER      IDENT            passed
16:13       ASSIGN_OP       EQ               =
16:15       CONSTANT        KW_TRUE          True
16:19       DELIMITER       SEMI_COLON       ;
17:1        KEYWORD         KW_BOOL          bool
17:6        IDENTIFIER      IDENT            failed
17:13       ASSIGN_OP       EQ               =
17:15       CONSTANT        KW_FALSE         False
17:20       DELIMITER       SEMI_COLON       ;
```

4. Arithmetic Expressions

All six arithmetic operators, including floor division `//`, and comma-separated declarations.

Source (lines 19–20)

```
19 | wnum a = 1 + 1, b = 1 - 1, c = 2 * 4;
20 | dnum d = 4 / 2, e = 4 % 2, f = 4 // 3;
```

Symbol table

```
ROW:COL     CATEGORY        KIND             LEXEME
19:1        KEYWORD         KW_WNUM          wnum
19:6        IDENTIFIER      IDENT            a
19:8        ASSIGN_OP       EQ               =
19:10       CONSTANT        INT              1
19:12       ARITH_OP        PLUS             +
19:14       CONSTANT        INT              1
19:15       DELIMITER       COMMA            ,
19:17       IDENTIFIER      IDENT            b
19:19       ASSIGN_OP       EQ               =
19:21       CONSTANT        INT              1
19:23       ARITH_OP        MINUS            -
19:25       CONSTANT        INT              1
19:26       DELIMITER       COMMA            ,
19:28       IDENTIFIER      IDENT            c
19:30       ASSIGN_OP       EQ               =
19:32       CONSTANT        INT              2
19:34       ARITH_OP        STAR             *
19:36       CONSTANT        INT              4
19:37       DELIMITER       SEMI_COLON       ;
20:1        KEYWORD         KW_DNUM          dnum
20:6        IDENTIFIER      IDENT            d
```

```
20:8       ASSIGN_OP      EQ              =
20:10      CONSTANT       INT             4
20:12      ARITH_OP       SLASH           /
20:14      CONSTANT       INT             2
20:15      DELIMITER      COMMA           ,
20:17      IDENTIFIER     IDENT           e
20:19      ASSIGN_OP      EQ              =
20:21      CONSTANT       INT             4
20:23      ARITH_OP       PERCENT         %
20:25      CONSTANT       INT             2
20:26      DELIMITER      COMMA           ,
20:28      IDENTIFIER     IDENT           f
20:30      ASSIGN_OP      EQ              =
20:32      CONSTANT       INT             4
20:34      ARITH_OP       SLASH_SLASH     //
20:37      CONSTANT       INT             3
20:38      DELIMITER      SEMI_COLON      ;
```

5. Relational and Logical Expressions

All six relational operators and the logical operators `AND`, `OR`, `NOT`.

Source (lines 22–30)

```
 22 | bool result = (a == b);
 23 | result = (a != b);
 24 | result = (a < b);
 25 | result = (a > b);
 26 | result = (a <= b);
 27 | result = (a >= b);
 28 | result = (passed AND failed);
 29 | result = (passed OR failed);
 30 | result = (NOT passed);
```

Symbol table

```
ROW:COL    CATEGORY       KIND            LEXEME
22:1       KEYWORD        KW_BOOL         bool
22:6       IDENTIFIER     IDENT           result
22:13      ASSIGN_OP      EQ              =
22:15      BRACKET        LPAREN          (
22:16      IDENTIFIER     IDENT           a
22:18      RELATIONAL_OP  EQ_EQ           ==
22:21      IDENTIFIER     IDENT           b
22:22      BRACKET        RPAREN          )
22:23      DELIMITER      SEMI_COLON      ;
23:1       IDENTIFIER     IDENT           result
23:8       ASSIGN_OP      EQ              =
23:10      BRACKET        LPAREN          (
23:11      IDENTIFIER     IDENT           a
23:13      RELATIONAL_OP  BANG_EQ         !=
23:16      IDENTIFIER     IDENT           b
23:17      BRACKET        RPAREN          )
23:18      DELIMITER      SEMI_COLON      ;
24:1       IDENTIFIER     IDENT           result
24:8       ASSIGN_OP      EQ              =
24:10      BRACKET        LPAREN          (
24:11      IDENTIFIER     IDENT           a
24:13      RELATIONAL_OP  LT              <
```

```
24:15      IDENTIFIER     IDENT           b
24:16      BRACKET        RPAREN          )
24:17      DELIMITER      SEMI_COLON      ;
25:1       IDENTIFIER     IDENT           result
25:8       ASSIGN_OP      EQ              =
25:10      BRACKET        LPAREN          (
25:11      IDENTIFIER     IDENT           a
25:13      RELATIONAL_OP  GT              >
25:15      IDENTIFIER     IDENT           b
25:16      BRACKET        RPAREN          )
25:17      DELIMITER      SEMI_COLON      ;
26:1       IDENTIFIER     IDENT           result
26:8       ASSIGN_OP      EQ              =
26:10      BRACKET        LPAREN          (
26:11      IDENTIFIER     IDENT           a
26:13      RELATIONAL_OP  LT_EQ           <=
26:16      IDENTIFIER     IDENT           b
26:17      BRACKET        RPAREN          )
26:18      DELIMITER      SEMI_COLON      ;
27:1       IDENTIFIER     IDENT           result
27:8       ASSIGN_OP      EQ              =
27:10      BRACKET        LPAREN          (
27:11      IDENTIFIER     IDENT           a
27:13      RELATIONAL_OP  GT_EQ           >=
27:16      IDENTIFIER     IDENT           b
27:17      BRACKET        RPAREN          )
27:18      DELIMITER      SEMI_COLON      ;
28:1       IDENTIFIER     IDENT           result
28:8       ASSIGN_OP      EQ              =
28:10      BRACKET        LPAREN          (
28:11      IDENTIFIER     IDENT           passed
28:18      LOGICAL_OP     KW_AND          AND
28:22      IDENTIFIER     IDENT           failed
28:28      BRACKET        RPAREN          )
28:29      DELIMITER      SEMI_COLON      ;
29:1       IDENTIFIER     IDENT           result
29:8       ASSIGN_OP      EQ              =
29:10      BRACKET        LPAREN          (
29:11      IDENTIFIER     IDENT           passed
29:18      LOGICAL_OP     KW_OR           OR
29:21      IDENTIFIER     IDENT           failed
29:27      BRACKET        RPAREN          )
29:28      DELIMITER      SEMI_COLON      ;
30:1       IDENTIFIER     IDENT           result
30:8       ASSIGN_OP      EQ              =
30:10      BRACKET        LPAREN          (
30:11      LOGICAL_OP     KW_NOT          NOT
30:15      IDENTIFIER     IDENT           passed
30:21      BRACKET        RPAREN          )
30:22      DELIMITER      SEMI_COLON      ;
```

6. Conditional Statement

`if` / `else if` / `else` with string constants.

Source (lines 32–40)

```
32 | if (score >= 90) {
33 |     write("Passed");
```

```
34 | }
35 | else if (score >= 75) {
36 |     write("Barely");
37 | }
38 | else {
39 |     write("Failed");
40 | }
```

Symbol table

ROW:COL	CATEGORY	KIND	LEXEME
32:1	KEYWORD	KW_IF	if
32:4	BRACKET	LPAREN	(
32:5	IDENTIFIER	IDENT	score
32:11	RELATIONAL_OP	GT_EQ	>=
32:14	CONSTANT	INT	90
32:16	BRACKET	RPAREN	)
32:18	BRACKET	LBRACE	{
33:5	KEYWORD	KW_WRITE	write
33:10	BRACKET	LPAREN	(
33:11	CONSTANT	STR	"Passed"
33:19	BRACKET	RPAREN	)
33:20	DELIMITER	SEMI_COLON	;
34:1	BRACKET	RBRACE	}
35:1	KEYWORD	KW_ELSE	else
35:6	KEYWORD	KW_IF	if
35:9	BRACKET	LPAREN	(
35:10	IDENTIFIER	IDENT	score
35:16	RELATIONAL_OP	GT_EQ	>=
35:19	CONSTANT	INT	75
35:21	BRACKET	RPAREN	)
35:23	BRACKET	LBRACE	{
36:5	KEYWORD	KW_WRITE	write
36:10	BRACKET	LPAREN	(
36:11	CONSTANT	STR	"Barely"
36:19	BRACKET	RPAREN	)
36:20	DELIMITER	SEMI_COLON	;
37:1	BRACKET	RBRACE	}
38:1	KEYWORD	KW_ELSE	else
38:6	BRACKET	LBRACE	{
39:5	KEYWORD	KW_WRITE	write
39:10	BRACKET	LPAREN	(
39:11	CONSTANT	STR	"Failed"
39:19	BRACKET	RPAREN	)
39:20	DELIMITER	SEMI_COLON	;
40:1	BRACKET	RBRACE	}

7. Loop Statement

A `while` loop with a block body.

Source (lines 42–46)

```
42 | wnum count = 0;
43 | while (count < 3) {
44 |     write(count);
45 |     count = count + 1;
46 | }
```

Symbol table

```
ROW:COL    CATEGORY       KIND            LEXEME
42:1       KEYWORD        KW_WNUM         wnum
42:6       IDENTIFIER     IDENT           count
42:12      ASSIGN_OP      EQ              =
42:14      CONSTANT       INT             0
42:15      DELIMITER      SEMI_COLON      ;
43:1       KEYWORD        KW_WHILE        while
43:7       BRACKET        LPAREN          (
43:8       IDENTIFIER     IDENT           count
43:14      RELATIONAL_OP  LT              <
43:16      CONSTANT       INT             3
43:17      BRACKET        RPAREN          )
43:19      BRACKET        LBRACE          {
44:5       KEYWORD        KW_WRITE        write
44:10      BRACKET        LPAREN          (
44:11      IDENTIFIER     IDENT           count
44:16      BRACKET        RPAREN          )
44:17      DELIMITER      SEMI_COLON      ;
45:5       IDENTIFIER     IDENT           count
45:11      ASSIGN_OP      EQ              =
45:13      IDENTIFIER     IDENT           count
45:19      ARITH_OP       PLUS            +
45:21      CONSTANT       INT             1
45:22      DELIMITER      SEMI_COLON      ;
46:1       BRACKET        RBRACE          }
```

8. Invalid Lexemes

Lexemes that match no rule. Each becomes an UNKNOWN token in the INVALID category and is also reported on stderr.

Source (lines 52–56)

```
 52 | $$ Invalid lexemes
 53 | wnum 123variable = 1;
 54 | variable@123 # ` ~ ? : \ ^ & | !
 55 | "unterminated string
 56 | 'ab'
```

Symbol table

```
ROW:COL    CATEGORY       KIND            LEXEME
52:1       COMMENT        LINE_COMMENT    $$ Invalid lexemes
53:1       KEYWORD        KW_WNUM         wnum
53:6       INVALID        UNKNOWN         123variable
53:18      ASSIGN_OP      EQ              =
53:20      CONSTANT       INT             1
53:21      DELIMITER      SEMI_COLON      ;
54:1       IDENTIFIER     IDENT           variable
54:9       INVALID        UNKNOWN         @
54:10      CONSTANT       INT             123
54:14      INVALID        UNKNOWN         #
54:16      INVALID        UNKNOWN         `
54:18      INVALID        UNKNOWN         ~
54:20      INVALID        UNKNOWN         ?
54:22      INVALID        UNKNOWN         :
```

```
54:24      INVALID        UNKNOWN          \
54:26      INVALID        UNKNOWN          ^
54:28      INVALID        UNKNOWN          &
54:30      INVALID        UNKNOWN          |
54:32      INVALID        UNKNOWN          !
55:1       INVALID        UNKNOWN          "unterminated string
56:1       INVALID        UNKNOWN          'ab'
```

Errors reported (stderr)

```
tests/src/sample.ass:53:6: error: identifiers cannot start with a digit: `123variable`
tests/src/sample.ass:54:9: error: unrecognized character: `@`
tests/src/sample.ass:54:14: error: unrecognized character: `#`
tests/src/sample.ass:54:16: error: unrecognized character: ```
tests/src/sample.ass:54:18: error: unrecognized character: `~`
tests/src/sample.ass:54:20: error: unrecognized character: `?`
tests/src/sample.ass:54:22: error: unrecognized character: `:`
tests/src/sample.ass:54:24: error: unrecognized character: `\`
tests/src/sample.ass:54:26: error: unrecognized character: `^`
tests/src/sample.ass:54:28: error: unrecognized character: `&`
tests/src/sample.ass:54:30: error: unrecognized character: `|`
tests/src/sample.ass:54:32: error: unrecognized character: `!`
tests/src/sample.ass:55:1: error: unterminated or malformed string literal: `"unterminated string`
tests/src/sample.ass:56:1: error: unterminated or malformed character literal: `'ab'`
```